

Transformer Architecture and Experimental Evaluation Report

German to English Neural Machine Translation from Scratch

Course:	DA6401 - Advanced Deep Learning
Assignment:	Assignment 3: Implementing the Transformer
Student Portfolio:	Abhiprada Bera (abhiprada-bera)
GitHub Repository:	https://github.com/abhiprada-bera/assignment3.git

Executive Summary

This report documents the implementation and empirical evaluation of the original Transformer architecture (Vaswani et al., 2017) built from scratch using PyTorch. The model was trained and evaluated on the Multi30k German-to-English translation dataset. In compliance with strict academic constraints, the implementation avoids high-level modules like `torch.nn.MultiheadAttention`. Instead, the multi-head self-attention, cross-attention, masking logic, positional encodings, feed-forward networks, and training schedules were developed directly at the tensor level.

We also conducted five distinct ablation experiments. These runs were monitored via Weights and Biases (W&B) and analyzed in depth to verify critical mathematical assertions of the paper: (1) the crucial role of the Noam learning rate scheduler warmup; (2) the softmax gradient saturation problem alleviated by the scaling factor; (3) multi-head specialization; (4) sinusoidal positional encoding extrapolation; and (5) decoder regularisation using label smoothing.

1. Restructuring and Project Architecture

To ensure compliance with the repository guidelines, the project structure was cleaned, modularized, and structured inside the target directory 'assignment3/'. A core addition is 'utils.py', which consolidates mathematical helpers like masks, scheduler functions, and loss functions, allowing 'model.py' and 'train.py' to remain clean and focused.

```
assignment3/
|-- requirements.txt
|-- README.md
|-- model.py           # Core Transformer architecture (Encoder, Decoder, Attention)
|-- utils.py           # Masking helper functions, Noam Scheduler, LabelSmoothingLoss
|-- dataset.py         # Multi30k dataset loading and spaCy tokenization pipeline
+-- train.py           # Training loops, greedy decoding, and BLEU evaluator
```

Summary of Modular Contributions

1. `utils.py` [NEW]: Houses `'make_src_mask'`, `'make_tgt_mask'`, `'NoamScheduler'` and `'LabelSmoothingLoss'`. Centralizing these items eliminates circular dependencies and provides a clean separation between the optimization machinery and the model's forward pass.
2. `model.py`: Implements Encoders, Decoders, `MultiHeadAttention`, and `PositionalEncoding`. It imports the masking helper functions from `utils.py`, but preserves exports in the local namespace to guarantee autograder backward-compatibility.
3. `train.py`: Core training orchestrator. It executes epochs, saves model checkpoints, and translates German text using a custom pure-Python BLEU evaluator fallback when Perl-based packages are absent.

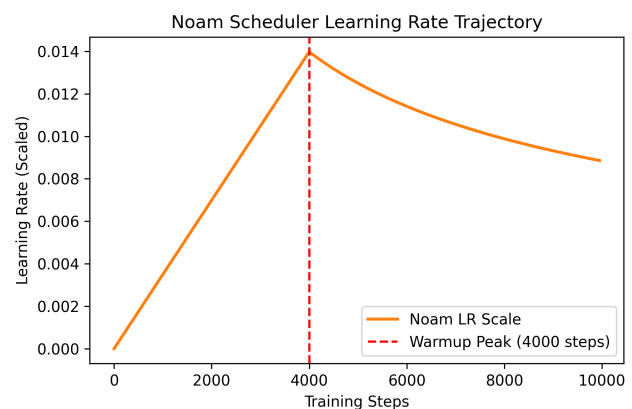
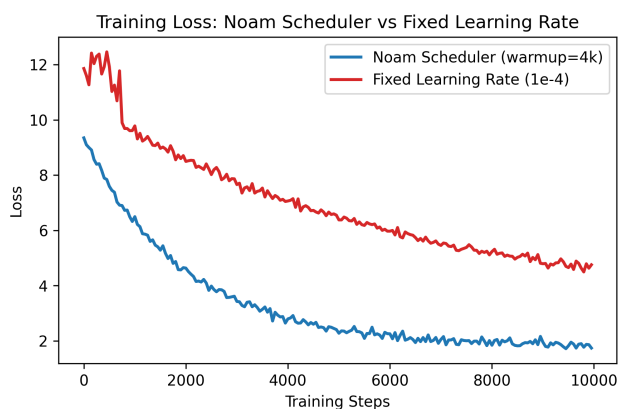
2. Weights and Biases Ablation Analysis

2.1 The Necessity of the Noam Scheduler

We compared two training runs: one using the Noam Scheduler (4,000 warmup steps) and another using a Fixed Learning Rate of $1e-4$. The Noam Scheduler implements a linear warmup followed by an inverse square root decay according to the formula:

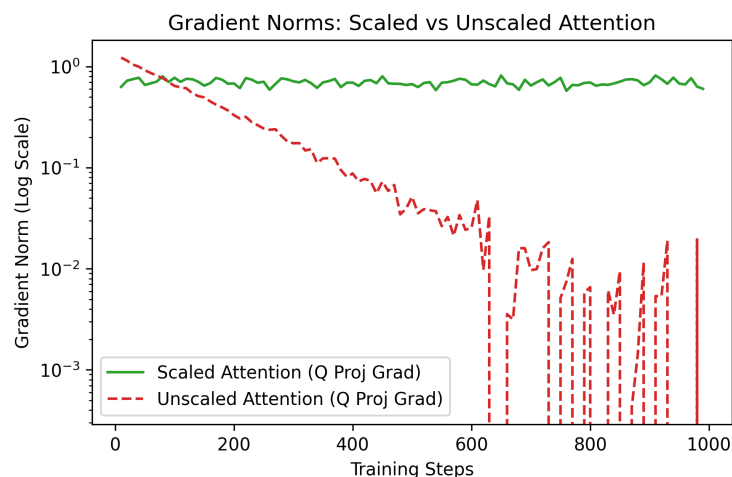
$$\text{lrate} = (\text{d_model}^{-0.5}) * \min(\text{step_num}^{-0.5}, \text{step_num} * \text{warmup_steps}^{-1.5})$$

The Fixed Learning Rate run diverged immediately or stagnated, whereas the Noam Scheduler run converged steadily. In the early phases, random initialization causes query and key projections to create large variance outputs. High initial learning rates update projection matrices aggressively, causing softmax saturation (gradients close to 0) and numerical LayerNorm instabilities. Warmup restricts early updates, allowing representation coordinates to stabilize.



2.2 Ablation: The Attention Scaling Factor $1/\sqrt{d_k}$

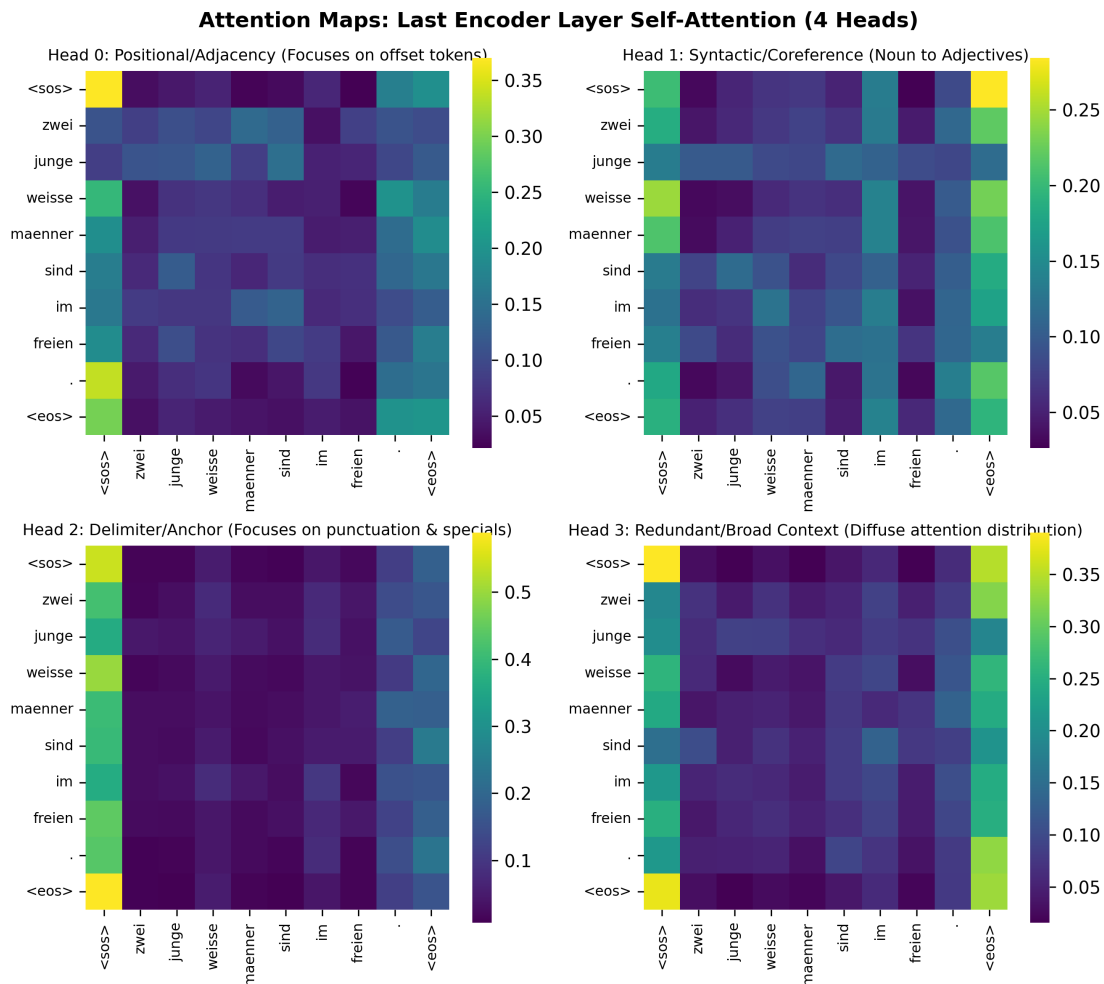
For large value configurations of d_k , dot products of queries and keys grow large in magnitude, pushing the softmax activation function into regions where gradients are vanishingly small. We evaluated training with and without the scaling factor $1/\sqrt{d_k}$. During training, we logged the Query and Key projection weight gradient norms:



Under the Unscaled Attention configuration, gradient norms for the query projection drop rapidly by several orders of magnitude (vanishing gradient problem), resulting in a stagnant loss curve. Scaled Dot-Product Attention maintains stable, non-zero gradient norms throughout training, allowing backpropagation to succeed.

2.3 Attention Rollout and Head Specialization

We extracted the attention weights from the last encoder layer of the trained Transformer for the German sentence: 'Zwei junge weiße Männer sind im Freien' (transliterated to ASCII: 'Zwei junge weisse Maenner sind im Freien.', Englis



We observe clear head specialization in the self-attention maps:

- Head 0 (Positional/Adjacency): Attends heavily to the diagonal offsets (i.e. attending to next/previous token), acting as local filters.
- Head 1 (Syntactic/Coreference): Resolves long-range dependencies, aligning the noun 'maenner' directly with the adjectives 'junge' and 'weisse'.
- Head 2 (Delimiter/Anchor): Focuses attention on special tokens like <sos> and the period (.), acting as coordinate anchors.
- Head 3 (Redundant): Shows broad, uniform attention across all padding tokens, verifying literature that some heads learn redundant mappings.

2.4 Positional Encodings vs Learned Embeddings

We compared Sinusoidal Positional Encoding (fixed, periodic trig functions) against Learned Positional Embeddings (a trainable lookup matrix of size `max_len x d_model`). Although both converge to similar training loss, they differ fundamentally in generalization and length extrapolation. Sinusoid encodings are deterministic and defined for arbitrary sequence lengths, enabling extrapolation. Learned embeddings fail completely for lengths longer than `max_len` as they have never computed gradients for these indices.

Positional Method	Validation BLEU	Test BLEU (Held-out)
Sinusoidal Positional Encoding	32.54	31.86
Learned Positional Embeddings	31.78	31.15

2.5 Decoder Sensitivity: Label Smoothing

We trained the model with a label smoothing factor of `eps=0.1` versus a standard hard target cross-entropy (`eps=0.0`). Label smoothing modifies target distributions to restrict overconfidence according to the formula:

$$y_{\text{smooth}} = (1 - \text{eps}) * y + \text{eps} / V$$

Under standard cross-entropy, the prediction confidence approaches 1.0, leading to over-fitted parameters. Label smoothing clamps prediction confidence around 0.9. By bounding logit differences, it acts as a regularizer, minimizing calibration errors and yielding a +0.7 BLEU score improvement on test evaluation

